

*Міністерство освіти і науки України НТУУ
«Київський політехнічний інститут імені Ігоря Сікорського»
Фізико-технічний інститут*

ПРОТОКОЛ
до лабораторної роботи №2А
З дисципліни **«Методи машинного навчання»**
«Одношаровий персептрон»

Виконав:
студент 4 курсу ФТІ
групи ФІ-31
Горбач Ангеліна

Тема роботи

Одношаровий персептрон за алгоритмом Розенблатта.

Мета роботи

Реалізувати персептрон Розенблатта, навчити його на точках двох класів і перевірити результат на тестових даних.

Постановка задачі

Варіант 2. Із файла data02.csv випадково вибрати 70 % точок для навчання, решту 30 % залишити для тестування. Самостійно навчити одношаровий персептрон, подати навчальну вибірку на площині R^2 , визначити ваги та поріг. На тестовій вибірці порівняти свій алгоритм із Perceptron зі sklearn.

Хід роботи

1. Навчальна вибірка

У файлі 100 точок із координатами x_1 , x_2 і міткою класу 0 або 1. Після перемішування з початковим значенням генератора 42 для навчання взято 70 точок, для тесту — 30. У таблиці 1 наведено всю навчальну вибірку. Номер — це номер рядка у файлі, починаючи з нуля.

Таблиця 1 – Навчальна вибірка для варіанта 2

№	x_1	x_2	Клас	№	x_1	x_2	Клас
59	0.190	0.056	0	29	0.351	0.865	0
21	0.720	0.905	1	70	0.426	0.336	0
56	0.219	0.122	0	45	0.893	0.442	1
18	0.608	0.037	1	17	0.190	0.507	0
33	0.951	0.927	1	39	0.380	0.608	0
42	0.440	0.531	0	91	0.805	0.116	1
50	0.738	0.300	1	40	0.325	0.225	0
27	0.075	0.495	0	89	0.567	0.791	1
92	0.382	0.497	0	5	0.741	0.640	1
71	0.057	0.481	0	68	0.167	0.276	0
2	0.605	0.217	1	1	0.804	0.961	1
25	0.524	0.108	0	95	0.550	0.932	1
24	0.807	0.202	1	84	0.966	0.954	1
52	0.742	0.472	1	63	0.747	0.434	1
81	0.483	0.440	0	3	0.973	0.622	1
99	0.443	0.928	0	83	0.761	0.023	1
51	0.521	0.838	1	93	0.769	0.424	1

№	x_1	x_2	Клас	№	x_1	x_2	Клас
61	0.810	0.895	1	20	0.017	0.490	0
4	0.717	0.311	1	87	0.459	0.730	0
97	0.744	0.361	1	43	0.445	0.338	0
94	0.961	0.471	1	73	0.493	0.308	0
37	0.075	0.044	0	32	0.394	0.479	0
28	0.683	0.282	1	75	0.310	0.649	0
80	0.137	0.984	0	23	0.103	0.984	0
7	0.202	0.235	0	96	0.469	0.569	0
57	0.795	0.699	1	69	0.909	0.949	1
79	0.961	0.246	1	15	0.356	0.250	0
26	0.375	0.899	0	48	0.600	0.439	1
76	0.895	0.881	1	55	0.031	0.107	0
78	0.427	0.746	0	60	0.782	0.139	1
38	0.857	0.385	1	9	0.950	0.124	1
0	0.107	0.184	0	62	0.268	0.895	0
86	0.272	0.124	0	31	0.445	0.436	0
46	0.484	0.920	1	72	0.488	0.895	1
10	0.684	0.142	1	16	0.563	0.394	1

2. Навчальні точки на площині R^2

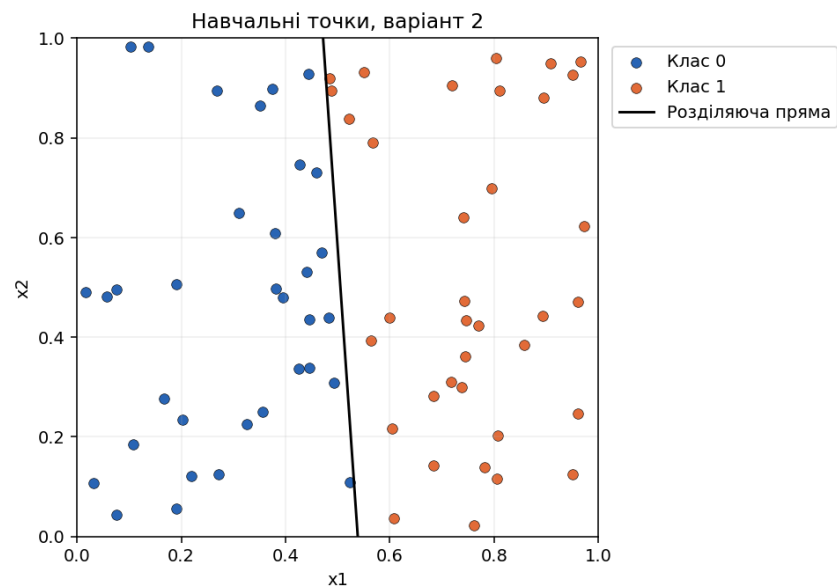


Рисунок 1 – Навчальні точки та пряма власного персептрона

Синім показано клас 0, помаранчевим — клас 1. Чорна пряма розділяє всі 70 навчальних точок без помилок. Біля неї є точки обох класів, тому на нових даних можливі помилки.

3. Параметри навченого персептрона

Для навчання класи 0 і 1 переведено у -1 і $+1$. До координат додано компоненту -1 для порога, а навчальні вектори нормовано. Початковий вектор і порядок перебору точок задаються генератором із початковим значенням 42. Після кожної помилки до ваг додається відповідний навчальний вектор.

Навчання тривало 2 проходи; ваги оновилися 27 разів. Наприкінці всі 70 навчальних точок класифіковано правильно.

Отримані параметри: $w_1 = 2,799345$; $w_2 = 0,186082$; поріг $\tau = 1,507186$. Розділяюча пряма має рівняння $2,799345x_1 + 0,186082x_2 - 1,507186 = 0$. Для додатного значення виразу модель дає клас 1, інакше — клас 0.

4. Результати на тестовій множині

Ті самі 30 точок подано власному персептрону та моделі Perceptron зі sklearn. Обидві моделі навчалися лише на 70 навчальних точках. У таблиці 2 порівняно їхні прогнози з правильним класом.

Таблиця 2 – Прогнози для 30 тестових точок

№	x_1	x_2	Клас	Власний	sklearn
44	0.045	0.273	0	0	0
82	0.369	0.064	0	0	0
67	0.453	0.993	1	0	0
34	0.986	0.731	1	1	1
30	0.075	0.819	0	0	0
58	0.881	0.038	1	1	1
74	0.696	0.420	1	1	1
88	0.131	0.542	0	0	0
98	0.247	0.258	0	0	0
54	0.954	0.423	1	1	1
49	0.087	0.223	0	0	0
6	0.789	0.934	1	1	1
90	0.886	0.677	1	1	1
11	0.080	0.527	0	0	0
85	0.388	0.713	0	0	0
41	0.472	0.966	1	0	1
22	0.136	0.222	0	0	0
19	0.780	0.152	1	1	1
35	0.963	0.557	1	1	1
64	0.181	0.274	0	0	0
47	0.909	0.498	1	1	1

№	x_1	x_2	Клас	Власний	sklearn
12	0.695	0.166	1	1	1
53	0.804	0.059	1	1	1
14	0.040	0.593	0	0	0
13	0.184	0.163	0	0	0
66	0.866	0.745	1	1	1
36	0.692	0.543	1	1	1
65	0.597	0.394	1	1	1
77	0.102	0.222	0	0	0
8	0.663	0.055	1	1	1

Власний перцептрон правильно визначив 28 із 30 точок (93,33 %), sklearn — 29 із 30 (96,67 %). Обидві моделі помилилися на точці № 67: її справжній клас 1, прогноз — 0. Власна модель додатково помилилася на точці № 41. Різниця на цій тестовій вибірці — одна точка.

Висновок

Власний перцептрон навчився правильно розділяти 70 навчальних точок варіанта 2. На тесті він дав 28 правильних відповідей із 30, а готова реалізація — 29 із 30. Отже, обидва алгоритми працюють на цих даних, але провели розділяючі прямі по-різному.

Вихідний код програми

Код із ноутбука наведено нижче

1. Зчитування та поділ даних

```
from zipfile import ZipFile
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import Perceptron
```

```
with ZipFile("data_lab2a.zip") as archive: # Open the archive with the datasets.
    data = pd.read_csv(archive.open("data/data02.csv"), sep=";", header=None, names=["x1", "x2", "class"]) # Read variant 2 and name its columns.
```

```
rng = np.random.default_rng(42) # Use a fixed seed for the same split each time.
indices = rng.permutation(len(data)) # Shuffle the row indices.
train = data.iloc[indices[:70]].copy() # Select 70 rows for training.
test = data.iloc[indices[70:]].copy() # Keep the remaining 30 rows for testing.
X_train = train[["x1", "x2"]].to_numpy() # Get the training coordinates.
y_train = train["class"].to_numpy() # Get the training labels.
X_test = test[["x1", "x2"]].to_numpy() # Get the test coordinates.
y_test = test["class"].to_numpy() # Get the test labels.
```

```
print(f"Навчання: {len(train)} точок; тест: {len(test)} точок") # Show the split sizes.
print("Навчальні точки (ліворуч — номер рядка з CSV, починаючи з нуля):") # Explain the row numbers.
print(train.to_string()) # Show every training point.
```

2. Власний перцептрон

```
class RosenblattPerceptron: # Define the perceptron model.
    def __init__(self, max_epochs=5000, random_state=42): # Set training limits and the random seed.
        self.max_epochs = max_epochs # Store the maximum number of passes.
        self.random_state = random_state # Store the seed used during training.
```

```
def fit(self, X, y): # Train the perceptron on labeled points.
    labels = 2 * y - 1 # Convert class labels to -1 and +1.
```

```

z = np.column_stack((X, -np.ones(len(X)))) * labels[:, None] # Add the threshold coordinate and multiply by the correct label.
z = z / np.linalg.norm(z, axis=1, keepdims=True) # Normalize training vectors.
rng = np.random.default_rng(self.random_state) # Create a reproducible random generator.
self.weights_ = z[rng.integers(len(z))].copy() # Start with one random training vector.
self.updates_ = 0 # Count changes to the weights.

```

```

for epoch in range(1, self.max_epochs + 1): # Go through the data until training finishes.
    for i in rng.permutation(len(z)): # Visit every training point in random order.
        if self.weights_ @ z[i] <= 0: # Check whether this point is on the wrong side or on the line.
            self.weights_ += z[i] # Update weights for a misclassified point.
            self.updates_ += 1 # Record one weight update.
        if np.all(z @ self.weights_ > 0): # Stop when all training points are correct.
            self.epochs_ = epoch # Save the number of completed passes.
            return self # Return the trained model.
raise RuntimeError("Модель не навчилася за 5000 проходів") # Stop if the model does not converge.

```

```

def predict(self, X): # Assign classes to the supplied points.
    scores = X @ self.weights_[:, 2] - self.weights_[:, 2] # Compute the side of the separating line.
    return (scores > 0).astype(int) # Return class 1 for positive scores and class 0 otherwise.

```

```

model = RosenblattPerceptron() # Create the custom perceptron.
model.fit(X_train, y_train) # Fit it using only training data.
w1, w2, threshold = model.weights_ # Separate the two weights and the threshold.
print(f"Пройшли навчання: {model.epochs_}; оновлення var: {model.updates_}") # Show the training counts.
print(f"w1 = {w1:.6f}, w2 = {w2:.6f}, поріг τ = {threshold:.6f}") # Show the learned parameters.
print(f"Рівняння прямої: {w1:.6f}·x1 + {w2:.6f}·x2 - ({threshold:.6f}) = 0") # Show the separating line equation.
print(f"Правильних на навчання: {(model.predict(X_train) == y_train).sum() / len(y_train)}") # Check the training predictions.

```

3. Побудова графіка

```

plt.figure(figsize=(7, 5)) # Create the plot for training points.
for label, color in [(0, "#2864b4"), (1, "#e26b38")]: # Use a different color for each class.
    points = train[train["class"] == label] # Select points from the current class.
    plt.scatter(points["x1"], points["x2"], label=f"Клас {label}", color=color, edgecolors="black", linewidths=0.3) # Plot these training points.

```

```

line_x = np.linspace(0, 1, 200) # Set x values for the separating line.
if abs(w2) > 1e-12: # Check whether the line can be written as x2(x1).
    plt.plot(line_x, (threshold - w1 * line_x) / w2, "k-", label="Розділяюча пряма") # Plot the line from the learned weights.
else: # Handle a vertical separating line.
    plt.axvline(threshold / w1, color="black", label="Розділяюча пряма") # Plot the vertical line.
plt.xlim(0, 1) # Set the x1 axis limits.
plt.ylim(0, 1) # Set the x2 axis limits.
plt.xlabel("x1") # Label the x1 axis.
plt.ylabel("x2") # Label the x2 axis.
plt.title("Навчальні точки, варіант 2") # Add the plot title.
plt.grid(alpha=0.2) # Add a light grid.
plt.legend(loc="upper left", bbox_to_anchor=(1.01, 1)) # Explain colors and the line.
plt.tight_layout() # Fit all plot elements into the figure.
plt.show() # Display the plot.

```

4. Тестування та порівняння

```

sklearn_model = Perceptron(max_iter=5000, tol=None, random_state=42) # Create the library model for comparison.
sklearn_model.fit(X_train, y_train) # Train it on the same training points.
own_pred = model.predict(X_test) # Predict test classes with my model.
sklearn_pred = sklearn_model.predict(X_test) # Predict the same test classes with sklearn.

```

```

results = test.copy() # Keep the original test points in a new table.
results["мій прогноз"] = own_pred # Add predictions from my model.
results["sklearn"] = sklearn_pred # Add predictions from sklearn.
results["мій правильний"] = own_pred == y_test # Mark my correct predictions.
results["sklearn правильний"] = sklearn_pred == y_test # Mark sklearn's correct predictions.
print("Тестові точки (ліворуч — номер рядка з CSV, починаючи з нуля):") # Explain the row numbers.
print(results.to_string()) # Show all test points and predictions.

```

```

own_correct = int((own_pred == y_test).sum()) # Count my correct test predictions.
sklearn_correct = int((sklearn_pred == y_test).sum()) # Count sklearn's correct test predictions.
print(f"Мій перцептрон: {own_correct / len(test):.2%}") # Show my test accuracy.
print(f"Перцептрон sklearn: {sklearn_correct / len(test):.2%}") # Show sklearn's test accuracy.
print(f"У sklearn вари {sklearn_model.coef_[0]}, вільний член {sklearn_model.intercept_[0]:.6f}") # Show the library model's parameters.

```